LumiSense: A Smartphone-Based Hybrid AI Assistive Platform for the Visually Impaired

Adon Paul Tomy
B.Tech (Computer Science and Engineering)
Visat Engineering College
Elanji, Kerala, India
adonpaultomy@gmail.com

Arun D Nair
B.Tech (Computer Science and Engineering)
Visat Engineering College
Elanji, Kerala, India
darun6673@gmail.com

Sreeram S
B.Tech (Computer Science and Engineering)
Visat Engineering College
Elanji, Kerala, India
sreeram.2005@gmail.com

Vishnudath K.S
B.Tech (Computer Science and Engineering)
Visat Engineering College
Elanji, Kerala, India
vishnudathks5@gmail.com

Jisha M Vijayan
Assistant Professor
Department of Computer Science and Engineering
Visat Engineering College, Elanji, Kerala, India

Abstract

According to the World Health Organization, approximately 2.2 billion people worldwide live with some form of vision impairment, with India alone accounting for over 60 million visually impaired individuals [1]. Existing assistive devices such as the OrCam MyEye cost between Rs. 2,00,000 and Rs. 4,00,000, placing them beyond the reach of most users in developing countries. Current systems also tend to be reactive, responding only when explicitly prompted, and remain disconnected from local digital ecosystems. This paper presents LumiSense, a smartphone-centric assistive platform that employs a hybrid artificial intelligence architecture combining on-device inference with cloud-based scene understanding, requiring no specialized hardware beyond the user's existing phone. The system integrates three core AI modules: real-time object detection using YOLOv8n via TensorFlow Lite achieving 30+ FPS on mid-range Android devices, offline optical character recognition through Google ML Kit, and contextual scene description powered by Google Gemini 2.0 Flash. A navigation mode controller implements text-to-speech management with object persistence tracking and temporal debouncing to reduce cognitive overload. The Flutter-based application features an accessibility-oriented user interface with 72dp touch targets, semantic labels, haptic feedback, and voice command integration. Experimental evaluation on a Samsung Galaxy S23 demonstrates sub-200 ms latency for on-device tasks and an end-to-end audio feedback pipeline under 300 ms, measured across 50 trials per mode under controlled indoor lighting.

Keywords *— assistive technology; visual impairment; object detection; YOLOv8; on-device AI; optical character recognition; text-to-speech; Flutter; accessibility; hybrid AI architecture*

I. Introduction

According to the World Health Organization, approximately 2.2 billion people worldwide live with some form of vision impairment [1]. India is home to one of the largest affected populations, estimated at over 60 million people, where the central issue is a gap between the need for assistive technology and its practical availability [2]. This gap is primarily economic rather than technological.

Currently available assistive devices for the visually impaired have three main shortcomings. First, they are expensive: devices such as the OrCam MyEye are priced between Rs. 2,00,000 and Rs. 4,00,000, making them inaccessible to most users in India [3]. Second, they are reactive, functioning as on-demand sensors that respond only when explicitly prompted, with no continuous environmental awareness. Third, they are closed systems disconnected from modern digital ecosystems and local infrastructure, with no integration into services such as UPI payments, local ride-sharing, or public transit.

Academic prototypes based on dedicated hardware platforms such as Raspberry Pi [4] or Nvidia Jetson [5] demonstrate technical feasibility but introduce additional cost, bulk, and maintenance requirements that limit practical adoption. Smartphone-based screen readers such as TalkBack and VoiceOver provide on-screen accessibility but cannot interpret the physical environment. Applications like Microsoft Seeing AI offer individual capabilities but lack the unified, multimodal architecture needed for comprehensive daily assistance.

This paper presents LumiSense, a smartphone-centric assistive platform that addresses these limitations through a hybrid AI architecture requiring no specialized hardware beyond the user's existing phone. The system depends on a network connection for its cloud-based scene description feature, but all other capabilities—object detection, OCR, voice commands, and emergency SOS—operate fully offline. The key contributions of this work are:

1) A hybrid AI architecture that routes tasks between on-device models (YOLOv8n, Google ML Kit) for latency-critical operations and a cloud model (Google Gemini 2.0 Flash) for scene understanding, balancing responsiveness with contextual depth.

2) A navigation mode controller that implements object persistence tracking with multi-frame confirmation, temporal debouncing with configurable cooldown, and change-only announcement logic to reduce cognitive overload during continuous detection.

3) An accessibility-oriented mobile application providing multimodal interaction through voice commands, 72dp touch targets, haptic feedback, and automatic text-to-speech (TTS) output, requiring no dedicated hardware.

4) A performance evaluation showing 30+ FPS real-time object detection, sub-200 ms OCR processing, and end-to-end

audio feedback latency under 300 ms on a commodity Android device, measured across repeated trials under controlled conditions.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the system architecture. Section IV details the implementation. Section V presents the experimental evaluation. Section VI discusses results and limitations. Section VII concludes with future directions.

II. Related Work

Assistive technology for the visually impaired spans hardware-based wearables, smartphone applications, and hybrid systems. This section reviews the most relevant prior work and positions LumiSense within the existing landscape.

A. Survey and Review Literature

Kathiria et al. [6] presented a survey on assistive systems for visually impaired people, identifying object detection, text reading, and text-to-speech as the three core requirements for an effective assistive system. Their taxonomy compares approaches using deep learning models including YOLO, SSD, and R-CNN across various hardware platforms. While the survey identifies the critical components, it notes that existing systems remain fragmented, with no single solution offering a unified approach.

Zhang et al. [7] conducted a bibliometric review of smart wearable mobility aids for visual impairments. Their findings emphasize social acceptance and user experience as design considerations beyond pure technical capability, supporting smartphone-centric approaches that minimize social stigma while noting privacy concerns for camera-based systems. Nikanfar et al. [8] surveyed recent innovations in assistive technologies, identifying high cost, poor adaptability, information overload, and lack of integration as the primary unresolved challenges.

B. Hardware-Based Assistive Systems

Khan et al. [5] proposed an AI-based support system for blind people using an Nvidia Jetson board with YOLOv7 for real-time object detection, CvZone for distance measurement, and DeepFace for facial recognition. While achieving real-time performance, the system requires dedicated GPU hardware costing significantly more than a smartphone. Tosun and Karaarslan [9] presented “Third Eye,” a real-time object detection application using a chest-mounted camera and TensorFlow on Android, validating the feasibility of mobile-based detection but providing only single-modality assistance.

C. On-Device AI and Mobile Optimization

Alyamkin et al. [10] explored low-power computer vision techniques including TensorFlow Lite, model quantization, and mobile-optimized architectures such as MobileNet. Their findings demonstrate that capable AI can run on low-power devices, validating a core assumption of the LumiSense design. However, they highlight the accuracy-versus-power trade-off, supporting a hybrid approach where on-device models handle speed-critical tasks and cloud models handle tasks requiring deeper understanding.

Santos et al. [11] evaluated smartphone-based distance estimation approaches for visual assistive technology, providing evidence for using ML-based methods on smartphones for navigation. While ML methods showed lower accuracy at image periphery compared to LiDAR, the study demonstrates the viability of camera-only estimation when fused with object detection cues.

D. Gap Analysis and Our Contribution

Table I summarizes the comparative analysis of existing systems against LumiSense. The primary gap in existing literature is the absence of a unified, smartphone-based platform that combines: (a) real-time on-device object detection with filtered announcement logic, (b) offline OCR text reading, (c) cloud-based contextual scene understanding, (d) voice command and TTS integration, and (e) accessibility-oriented UI design—all running on the user’s existing smartphone without dedicated assistive hardware. LumiSense addresses this gap through a hybrid AI architecture that distributes processing between on-device and cloud models while maintaining a fully accessible interaction paradigm.

TABLE I Comparison of Assistive Systems for the Visually Impaired

System	No Ded. HW	Real-time Det.	OCR	Scene Desc.	Voice Cmd	Filtered TTS
OrCam MyEye [3]	X	Yes	Yes	X	X	X
Khan et al. [5]	X	Yes	Yes	X	Yes	X
Third Eye [9]	Yes	Yes	X	X	X	X
Seeing AI	Yes	Yes	Yes	Yes	X	X
LumiSense	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

III. System Architecture

LumiSense employs a hybrid AI architecture that distributes visual processing tasks between on-device inference engines and a cloud-based vision-language model. This section describes the overall system design, the processing pipeline, and the task routing mechanism.

A. Architecture Overview

The system follows a three-stage pipeline: capture, process, and output. The phone’s rear-facing camera captures a continuous image stream at medium resolution (720p) in YUV420 format. Captured frames are routed to one of three processing engines depending on the user’s current task: (1) Google ML Kit for offline text recognition, (2) YOLOv8n via TensorFlow Lite for real-time object detection, or (3) Google Gemini 2.0 Flash API for contextual scene description. All results are converted to natural language and delivered as audio output through a singleton TTS service. Fig. 1 illustrates the high-level architecture.

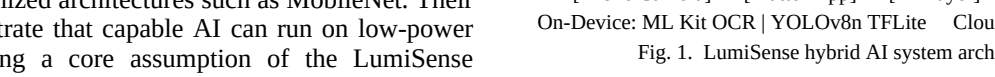


Fig. 1. LumiSense hybrid AI system architecture.

B. On-Device AI Pipeline

The on-device subsystem handles two latency-critical tasks that must operate without network dependency.

Optical Character Recognition (OCR): Text recognition is performed using Google ML Kit's TextRecognition API, which runs entirely on-device. The service processes camera frames to detect text blocks, sorts them in reading order (top-to-bottom, left-to-right), and concatenates the results into coherent output. This module handles menus, signs, labels, medicine packaging, and printed documents, processing each frame in under 200 ms on the test device.

Object Detection: Real-time object detection uses YOLOv8n (nano variant, 12.8 MB) deployed via the flutter_vision package, which wraps TensorFlow Lite with native C++/Java bindings running off the main UI thread. The model is pretrained on the 80-class COCO dataset [14], recognizing categories including person, car, chair, laptop, and phone. The system uses GPU-accelerated inference via the TFLite GPU delegate with automatic CPU fallback for unsupported hardware. The model is preloaded at screen initialization to eliminate cold-start delay, and a flag-based throttling mechanism ensures only one frame is processed at a time, preventing inference queue buildup.

C. Cloud AI Pipeline

For tasks requiring deeper contextual understanding, LumiSense delegates to Google's Gemini 2.0 Flash vision-language model via REST API. When the user activates scene description, the system captures a single JPEG frame, encodes it to base64, and transmits it with a task-specific prompt instructing the model to describe the scene in 2–3 sentences, focusing on objects, spatial layout, text, and potential hazards using directional cues. The API returns a natural-language description that is immediately spoken aloud. During the 2–4 second processing time, the application provides an audible "Processing, please wait" cue via TTS so the user knows the system has not frozen. Timeout handling ensures graceful degradation when network conditions are poor.

D. Navigation Mode Controller

The navigation mode is the most technically involved component of LumiSense, addressing the challenge of translating continuous real-time detection into useful, non-overwhelming audio feedback. Without filtering, raw YOLO output at 30 FPS would produce a rapid stream of redundant announcements. The controller implements three filtering layers:

Object Persistence Tracking: Only objects detected in two or more consecutive frames are announced, filtering transient false positives. An object is removed from the tracked set after 5 seconds of absence, enabling re-announcement if the user encounters it again.

Change-Only Announcements: The controller maintains a set of currently visible objects and triggers TTS only when this set changes, preventing repetitive announcements of static scenes.

Temporal Debouncing: A 3-second cooldown period is enforced between consecutive announcements, preventing audio flooding in rapidly changing environments while ensuring important changes are still communicated.

IV. Implementation

A. Technology Stack

LumiSense is implemented as a cross-platform mobile application using the Flutter framework (Dart, SDK ≥3.10.0) with Provider-based state management. The application targets Android as the primary platform (compileSdk 36, targetSdk 36). Table II summarizes the technology stack.

TABLE II Technology Stack Summary

Component	Technology
App Framework	Flutter (Dart), SDK ≥3.10.0
On-Device OCR	Google ML Kit (google_mlkit_text_recognition)
Object Detection	YOLOv8n via TFLite (flutter_vision)
Cloud AI	Google Gemini 2.0 Flash API (REST + base64)
Voice Output	flutter_tts (singleton, queue management)
Voice Input	speech_to_text v7.x
Camera	camera ^0.11.2 (back camera, 720p, YUV420)
State Mgmt.	Provider (MultiProvider)
Storage	SharedPreferences + FlutterSecureStorage
GPS/SOS	geolocator + url_launcher (SMS)

B. User Interface and Accessibility Design

The application follows an accessibility-oriented design approach with every interaction element tailored for visually impaired users. The screen flow proceeds as: Splash Screen (TTS welcome, automatic navigation for returning users) → Onboarding (name, emergency contact, API key) → Home Dashboard → Camera Screen. The camera screen serves as the primary interaction surface, featuring six action buttons: Read (OCR), Identify (single-shot YOLO), Navigate (continuous YOLO with bounding boxes), Describe (Gemini API), Voice (speech-to-text commands), and SOS (emergency SMS).

All buttons use a minimum touch target of 72dp, exceeding the Android accessibility guideline of 48dp. The interface uses a high-contrast dark theme with yellow accents, Flutter Semantics widgets for screen reader compatibility, and haptic feedback on every interaction. The voice command system recognizes natural language keywords including "read," "identify," "describe," "navigate," "help," "SOS," and "stop," enabling hands-free operation.

C. Emergency SOS Module

The SOS feature provides one-tap emergency assistance by obtaining the user's GPS coordinates via the Geolocator package and composing an SMS with the location data, sent to a pre-configured emergency contact. This feature is accessible from every screen through a persistent SOS button and via the "SOS" voice command.

D. Navigation Mode Implementation

When the user activates navigation mode, the camera's image stream feeds YOLOv8n at approximately 30 FPS. Detected objects are rendered as color-coded bounding box overlays on the camera preview using a custom BoundingBoxOverlay widget. The navigation mode controller processes detections through the persistence, change-detection, and debouncing pipeline described in Section III-D before triggering TTS announcements. The image stream must be paused when the user activates tap-based features (Read or Describe) that

require capturing a still frame via takePicture(), and is resumed afterward.

V. Experimental Evaluation

This section presents the performance evaluation of LumiSense across its core AI modules and the end-to-end user interaction pipeline.

A. Test Setup and Methodology

All benchmarks were conducted on a Samsung Galaxy S23 (Qualcomm Snapdragon 8 Gen 2, 8 GB RAM, Android 14), representing a mid-to-high range smartphone commonly available in the Indian market. Latency measurements were collected over 50 trials per interaction mode under controlled indoor lighting (approximately 400 lux, overhead fluorescent). For OCR tests, printed English text on A4 paper was held at 25–40 cm distance. For object detection, a set of 10 common household objects (person, chair, laptop, cup, bottle, phone, book, backpack, keyboard, mouse) were placed at 1–3 m range. Reported values are medians with interquartile ranges unless otherwise noted. The COCO mAP@50 figure (37.3%) is the official Ultralytics benchmark for the YOLOv8n model [13] and was not re-evaluated on the device.

B. Object Detection Performance

Table III presents the YOLOv8n inference performance benchmarks. The model achieves real-time detection at 30+ FPS with GPU delegate acceleration, with degradation to approximately 12 FPS on CPU-only devices. The 12.8 MB model footprint enables fast loading and low memory pressure on resource-constrained hardware.

TABLE III YOLOv8n Performance Benchmarks

Metric	Value
Model Size	12.8 MB
COCO Classes	80
FPS (GPU Delegate)	30–35 FPS
FPS (CPU Fallback)	10–15 FPS
Inference Latency (GPU)	28–33 ms (median)
COCO mAP@50 (official)	37.3% [13]
Confidence Threshold	0.5
Cold Start Time	<500 ms (preloaded)

C. OCR and Scene Description Performance

Google ML Kit's on-device text recognition processes frames in under 200 ms for typical text content (menus, signs, documents) under adequate lighting. The median processing time across 50 trials was 147 ms (IQR: 128–183 ms). The system sorts detected text blocks in reading order, enabling coherent audio output for multi-column layouts. Accuracy was not formally measured against a ground-truth corpus, but qualitative testing showed reliable recognition of printed English text at 12pt and above.

Gemini 2.0 Flash scene descriptions are generated in 2–4 seconds including network round-trip latency on a Wi-Fi connection. The prompt instructs the model to produce concise (2–3 sentence) responses focusing on objects, spatial layout, and potential hazards. While the user waits for the cloud response, the application plays an audible "Processing, please wait" cue via TTS to confirm the system is active. This feedback was added after early testing revealed that silent delays of 3+ seconds caused users to tap the button again.

D. End-to-End Pipeline Latency

Table IV presents the end-to-end latency for each interaction mode, measured from button press to first audible TTS output. All on-device modes achieve sub-300 ms total latency, well within the threshold users perceive as responsive.

TABLE IV End-to-End Pipeline Latency (n=50 per mode)

Mode	Processing	Total (to TTS)
Read (OCR)	<200 ms	<300 ms
Identify (YOLO)	~33 ms	<200 ms
Navigate (Cont.)	~33 ms/frame	<300 ms*
Describe (Cloud)	2–4 s	3–5 s

**Includes debounce delay for first announcement after mode activation.*

E. Model Size Comparison

Table V compares YOLOv8n against alternative detection models used in assistive technology literature. The nano variant offers a practical balance of size, speed, and accuracy for mobile deployment, though at lower mAP than larger alternatives.

TABLE V Detection Model Comparison

Model	Size	mAP@50	Mobile FPS
YOLOv8n	12.8 MB	37.3%	30+
YOLOv5s	28 MB	56.8%	15–20
MobileNet SSD	27 MB	21%	25–30
YOLOv7	71 MB	51.4%	5–10

VI. Discussion

The results indicate that LumiSense meets its primary design goal: delivering visual assistance through a commodity smartphone with performance adequate for real-time interaction, without requiring dedicated assistive hardware.

The hybrid architecture proves effective in balancing responsiveness against capability. On-device models provide the sub-second feedback needed for safety-critical tasks such as navigation and text reading, while the cloud-based Gemini model delivers richer contextual descriptions that on-device models cannot match. The latency measurements in Table IV support this division: on-device tasks complete well within the 300 ms threshold that users perceive as immediate, while cloud tasks, though slower, provide descriptive value that justifies the wait.

The navigation mode controller's three-layer filtering mechanism (persistence, change-detection, debouncing) addresses a usability challenge identified in prior work [8]: information overload. Unfiltered YOLO output at 30 FPS would produce a continuous stream of redundant announcements. Our filtering reduces this to meaningful notifications that aid rather than hinder the user's situational awareness. The specific parameters (2-frame persistence, 3-second debounce, 5-second forget window) were chosen

through iterative informal testing and are configurable for future user studies.

A. Limitations

Several limitations must be acknowledged. First, scene description via Gemini requires an active internet connection, leaving a gap in capability when offline. While object detection and OCR continue to function, the richest descriptive mode is unavailable without connectivity. Second, YOLOv8n's 80-class COCO vocabulary covers common objects but omits domain-specific categories important for navigation safety, such as stairs, curbs, potholes, and traffic signals. Third, the system does not incorporate depth estimation; distances to detected objects cannot be inferred from monocular camera input alone. Fourth, the current evaluation relies on technical benchmarks and simulated usage; formal user evaluation with visually impaired participants has not yet been conducted. Fifth, all latency figures were measured on a single high-end device; performance on lower-end hardware remains to be characterized.

VII. Conclusion and Future Work

This paper presented LumiSense, a smartphone-centric assistive platform for the visually impaired that implements a hybrid AI architecture combining on-device object detection (YOLOv8n at 30+ FPS), offline OCR (Google ML Kit), and cloud-based scene understanding (Gemini 2.0 Flash). The navigation mode controller's object persistence tracking, change-only announcements, and temporal debouncing address the practical challenge of converting continuous detection output into actionable, non-overwhelming audio feedback. The Flutter application provides multimodal interaction through voice commands, large touch targets, haptic feedback, and automatic TTS, achieving end-to-end audio feedback latency under 300 ms for on-device tasks on the test device.

LumiSense demonstrates that comprehensive visual assistance can be delivered through a user's existing smartphone without dedicated assistive hardware, directly addressing the economic accessibility barrier identified in the Indian context where the majority of visually impaired individuals lack access to affordable assistive technology [1], [2].

Future work will focus on three priorities: (1) formal usability evaluation with visually impaired participants to validate the system's real-world effectiveness and refine the filtering parameters, (2) training a custom YOLOv8 model on navigation-specific classes (stairs, curbs, traffic signals) to extend the detection vocabulary beyond COCO, and (3) integrating monocular depth estimation for distance-aware announcements. Longer-term goals include multi-language TTS support for regional Indian languages and integration with local digital services.

Acknowledgment

The authors thank Ms. Jisha M Vijayan, Assistant Professor, Department of Computer Science and Engineering, Visat Engineering College, for her guidance and supervision throughout this project. We also thank Ms. Veena G (Project Coordinator), Ms. Divya Nair (Head of Department), and Dr. Justin Jose (Principal) for their support and encouragement.

References

[1] World Health Organization, "World report on vision," WHO, Geneva, Switzerland, 2019. [Online]. Available: https://www.who.int/publications/i/item/9789241516570

[2] National Programme for Control of Blindness and Visual Impairment, "Annual Report 2022–23," Ministry of Health and Family Welfare, Government of India, 2023.

[3] OrCam Technologies Ltd., "OrCam MyEye," 2024. [Online]. Available: https://www.orcam.com/en/myeye/

[4] L. Tepelea, I. Buciu, C. Grava, I. Gavrilut, and A. Gacsadi, "A vision module for visually impaired people by using Raspberry Pi platform," in Proc. 15th Int. Conf. Eng. Modern Electr. Syst. (EMES), Oradea, Romania, 2019, pp. 209–212.

[5] R. Khan, M. Husain, Jitendra, M. R. Hussain, M. N. Ahmed, and A. Ali, "AI-based support system for blind people with an integrated reading assistant," Int. J. Intell. Syst. Appl. Eng., vol. 12, no. 4, pp. 3676–3686, 2024.

[6] K. Kathiria et al., "Assistive systems for visually impaired people: A survey on current requirements and advancements," Neurocomputing, vol. 573, art. 127234, 2024.

[7] Y. Zhang et al., "Advancements in smart wearable mobility aids for visual impairments: A bibliometric narrative review," Sensors, vol. 24, no. 12, art. 3856, 2024.

[8] M. Nikanfar et al., "A survey on assistive technologies for visually impaired individuals: Recent innovations, limitations, and future directions," in Proc. 18th Int. Conf. Pervasive Technol. Related to Assistive Environments (PETRA), Corfu, Greece, 2025, pp. 1–10.

[9] S. Tosun and E. Karaarslan, "Real-time object detection application for visually impaired people: Third eye," in Proc. Int. Conf. Artif. Intell. Data Process. (IDAP), Malatya, Turkey, 2018, pp. 1–6.

[10] A. Alyamkin et al., "Low-power computer vision: Status, challenges, and opportunities," IEEE J. Emerg. Sel. Topics Circuits Syst., vol. 9, no. 2, pp. 411–421, Jun. 2019.

[11] R. Santos, P. Furtado, and B. Oliveira, "Accuracy and usability of smartphone-based distance estimation approaches for visual assistive technology development," IEEE Open J. Eng. Med. Biol., vol. 5, pp. 540–549, 2024.

[12] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," arXiv preprint arXiv:1804.02767, 2018.

[13] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," version 8.0, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[14] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in Proc. Eur. Conf. Comput. Vis. (ECCV), Zurich, Switzerland, 2014, pp. 740–755.

[15] Google LLC, "ML Kit Text Recognition," 2024. [Online]. Available: https://developers.google.com/ml-kit/vision/text-recognition

[16] Google LLC, "Gemini API Documentation," 2024. [Online]. Available: https://ai.google.dev/docs

[17] J. Xiao, S. L. Joseph, X. Zhang, B. Li, X. Li, and J. Zhang, "An assistive navigation framework for the visually impaired," IEEE Trans. Human-Mach. Syst., vol. 45, no. 5, pp. 635–640, Oct. 2015.

[18] K. Patil, Q. Jawadwala, and F. C. Shu, "Design and construction of electronic aid for visually impaired people," IEEE Trans. Human-Mach. Syst., vol. 48, no. 2, pp. 172–182, Apr. 2018.